

Sample Outputs

1

```
le = LabelEncoder()
data = data.replace(r'\s$', np.nan, regex=True)
data["gender"] = le.fit_transform(data["gender"])
data["Partner"] = le.fit_transform(data["Partner"])
data["Dependents"] = le.fit_transform(data["Dependents"])
data["PhoneService"] = le.fit_transform(data["PhoneService"])
data["MultipleLines"] = le.fit_transform(data["MultipleLines"])
data["InternetService"] = le.fit_transform(data["InternetService"])
data["OnlineSecurity"] = le.fit_transform(data["OnlineSecurity"])
data["DeviceProtection"] = le.fit_transform(data["DeviceProtection"])
data["TechSupport"] = le.fit_transform(data["TechSupport"])
data["StreamingTV"] = le.fit_transform(data["StreamingTV"])
data["StreamingMovies"] = le.fit_transform(data["StreamingMovies"])
data["PaperlessBilling"] = le.fit_transform(data["PaperlessBilling"])
data["PaymentMethod"] = le.fit_transform(data["PaymentMethod"])
data["Churn"] = le.fit_transform(data["Churn"])
data["Contract"] = le.fit_transform(data["Contract"])
data["OnlineBackup"] = le.fit_transform(data["OnlineBackup"])
data = data.fillna(0) #null

data
```

```
'''
    gender SeniorCitizen Partner Dependents tenure PhoneService MultipleLines InternetService OnlineSecurity OnlineBackup DeviceProtection TechSupport StreamingTV StreamingMovies Contract PaperlessBilling Pa
0      0      0      0      1      0      1      0      1      0      0      2      0      0      0      0      0      1
1      1      1      0      0      0     34      1      0      0      2      0      2      0      0      0      1      0
2      1      0      0      0      2      1      0      0      2      2      0      0      0      0      0      0      1
3      1      0      0      0     45      0      1      0      2      0      2      2      0      0      0      1      0
4      0      0      0      0      2      1      0      1      0      0      0      0      0      0      0      0      1
...    ...    ...    ...    ...    ...    ...    ...    ...    ...    ...    ...    ...    ...    ...    ...    ...
7038    1      0      1      1     24      1      2      0      2      0      2      2      2      2      2      1      1
7039    0      0      1      1     72      1      2      1      0      2      2      0      2      2      2      1      1
7040    0      0      1      1     11      0      1      0      2      0      0      0      0      0      0      0      1
7041    1      1      1      0      4      1      2      1      0      0      0      0      0      0      0      0      1
7042    1      0      0      0     66      1      0      1      2      0      2      2      2      2      2      2      1
7043 rows x 20 columns
```

X/Y (Labels)

1

```
X = data.drop("MonthlyCharges", axis=1)
y = data["MonthlyCharges"]
len(X)
```

7043

Split Data

1

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
len(X_train), len(X_test)
```

(5634, 1409)

```

scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

display(pd.DataFrame(X_train_scaled, columns=X_train.columns, index=X_train.index).head())
display(pd.DataFrame(X_test_scaled, columns=X_test.columns, index=X_test.index).head())

```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies
2142	0.0	0.0	0.0	1.0	0.291667	1.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0	1.0
1623	0.0	0.0	0.0	0.0	0.750000	1.0	1.0	0.5	0.0	1.0	0.0	0.0	1.0	1.0
6074	1.0	0.0	1.0	0.0	0.013889	0.0	0.5	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1362	1.0	0.0	0.0	0.0	0.055556	1.0	0.0	0.5	0.0	0.0	0.0	0.0	0.0	0.0
6754	1.0	0.0	0.0	1.0	0.000000	1.0	1.0	0.0	1.0	1.0	0.0	1.0	0.0	0.0
	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies
185	0.0	0.0	1.0	0.0	0.013889	0.0	0.5	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2715	1.0	0.0	0.0	0.0	0.569444	1.0	1.0	1.0	0.5	0.5	0.5	0.5	0.5	0.5
3825	0.0	0.0	1.0	1.0	0.722222	1.0	0.0	1.0	0.5	0.5	0.5	0.5	0.5	0.5
1807	0.0	0.0	0.0	0.0	0.013889	1.0	0.0	0.5	0.0	0.0	1.0	0.0	0.0	0.0
132	1.0	0.0	0.0	0.0	0.930556	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0

```

model = LinearRegression()
model.fit(X_train, y_train)
preds = model.predict(X_test)
mse = mean_squared_error(y_test, preds)
r2 = r2_score(y_test, preds)
joblib.dump(model, 'linear_model.pkl')
print("MSE:", mse)
print("R² Score:", r2)

```

```

MSE: 175.2206794013519
R² Score: 0.8064661026095352

```

KNN

```

from sklearn.neighbors import KNeighborsRegressor

model = KNeighborsRegressor(n_neighbors=5, weights="distance")
model.fit(X_train, y_train)
preds = model.predict(X_test)

mse = mean_squared_error(y_test, preds)
r2 = r2_score(y_test, preds)
joblib.dump(model, 'KNN.pkl')
print("MSE:", mse)
print("R² Score:", r2)

```

```

MSE: 56.376628304547026
R² Score: 0.9377311591600392

```

```

from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(max_depth=5, random_state=42)
model.fit(X_train, y_train)
preds = model.predict(X_test)

mse = mean_squared_error(y_test, preds)
r2 = r2_score(y_test, preds)
joblib.dump(model, 'DecisionTreeRegressor.pkl')
print("MSE:", mse)
print("R² Score:", r2)

```

```

MSE: 28.084939416153833
R² Score: 0.968979758543606

```

```

# ===== GUI =====
import tkinter as tk
from tkinter import messagebox
import numpy as np
import pandas as pd
import joblib

# -----
# 1. LOAD MODELS
# -----
models = {}
try:
    models["Decision Tree"] = joblib.load("DecisionTreeRegressor.pkl")
    models["KNN"] = joblib.load("KNN.pkl")
    models["Linear Regression"] = joblib.load("linear_model.pkl")
    scaler = joblib.load("scaler.pkl")
except Exception as e:
    print(f"Warning: Files missing. {e}")

# -----
# Encoding Logic
# -----
def encode_value(label, value):
    value = str(value).lower().strip()
    yes_no = {"yes": 1, "no": 0}
    gender = {"male": 1, "female": 0}
    internet = {"dsl": 0, "fiber optic": 1, "no": 2}
    contract = {"month-to-month": 0, "one year": 1, "two year": 2}
    payment = {"electronic check": 0, "mailed check": 1, "bank transfer": 2, "credit card": 3}
    multi_lines = {"no": 0, "no phone service": 1, "yes": 2}
    internet_dependent = {"no": 0, "no internet service": 1, "yes": 2}

    if label == "gender": return gender[value]
    if label in ["Partner", "Dependents", "PhoneService", "PaperlessBilling", "Churn"]: return yes_no[value]
    if label == "MultipleLines": return multi_lines[value]
    if label in ["OnlineSecurity", "OnlineBackup", "DeviceProtection", "TechSupport", "StreamingTV", "StreamingMovies"]: return internet_dependent[value]
    if label == "InternetService": return internet[value]
    if label == "Contract": return contract[value]
    if label == "PaymentMethod": return payment[value]
    if label in ["SeniorCitizen", "tenure", "TotalCharges"]: return float(value)

    raise ValueError(f"Unknown label/value: {label} = {value}")

# -----
# Prediction Logic
# -----
def predict_price():
    try:
        model_name = model_var.get()
        active_model = models[model_name]

        input_values = []
        for label, entry in zip(labels, entries):
            input_values.append(encode_value(label, entry.get()))

        input_df = pd.DataFrame([input_values], columns=labels)

        if model_name == "KNN":
            final_input = scaler.transform(input_df)
        else:
            final_input = input_df

        prediction = active_model.predict(final_input)[0]
        messagebox.showinfo("Prediction", f"Model: {model_name}\nMonthly Charges: ${prediction:,.2f}")

    except Exception as e:
        messagebox.showerror("Error", f"Invalid input: {e}")

```

```

# GUI GRID LAYOUT
# -----
root = tk.Tk()
root.title("Churn Predictor")
root.geometry("700x550") # Set a fixed reasonable size

# 1. Header & Model Selection
header_frame = tk.Frame(root)
header_frame.pack(pady=10)

tk.Label(header_frame, text="Select Model:", font=("Arial", 10, "bold")).pack(side=tk.LEFT, padx=5)
model_names = list(models.keys()) if models else ["No Models Found"]
model_var = tk.StringVar(value=model_names[0])
tk.OptionMenu(header_frame, model_var, *model_names).pack(side=tk.LEFT)

# 2. Main Input Grid
# We use a specific frame for inputs to center them nicely
input_frame = tk.Frame(root)
input_frame.pack(pady=10, padx=10)

labels = [
    "gender", "SeniorCitizen", "Partner", "Dependents", "tenure",
    "PhoneService", "MultiLine", "InternetService", "OnlineSecurity",
    "OnlineBackup", "DeviceProtection", "TechSupport", "StreamingTV",
    "StreamingMovies", "Contract", "PaperlessBilling", "PaymentMethod",
    "TotalCharges", "Churn"
]

entries = []
options = {
    "gender": ["male", "female"],
    "Partner": ["yes", "no"],
    "Dependents": ["yes", "no"],
    "PhoneService": ["yes", "no"],
    "MultiLine": ["no", "no phone service", "yes"],
    "InternetService": ["dsl", "fiber optic", "no"],
    "OnlineSecurity": ["yes", "no", "no internet service"],
    "OnlineBackup": ["yes", "no", "no internet service"],
    "DeviceProtection": ["yes", "no", "no internet service"],
    "TechSupport": ["yes", "no", "no internet service"],
    "StreamingTV": ["yes", "no", "no internet service"],
    "StreamingMovies": ["yes", "no", "no internet service"],
    "Contract": ["month-to-month", "one year", "two year"],
    "PaperlessBilling": ["yes", "no"],
    "PaymentMethod": ["electronic check", "mailed check", "bank transfer", "credit card"],
    "Churn": ["yes", "no"]
}

# We loop through labels and place them in 2 columns (Row 0 to 9)
for index, label in enumerate(labels):
    # Calculate row and column
    # Even numbers go to column 0, Odd numbers go to column 2
    row = index // 2
    col_start = (index % 2) * 2

    # Label
    tk.Label(input_frame, text=label, font=("Arial", 9)).grid(row=row, column=col_start, sticky="w", padx=5, pady=5)

    # Entry / Dropdown
    if label in options:
        var = tk.StringVar(value=options[label][0])
        menu = tk.OptionMenu(input_frame, var, *options[label])
        menu.config(width=15) # Set fixed width for neatness
        menu.grid(row=row, column=col_start+1, sticky="w", padx=5, pady=5)
        entries.append(var)
    else:
        entry = tk.Entry(input_frame, width=15)
        entry.grid(row=row, column=col_start+1, sticky="w", padx=5, pady=5)
        entries.append(entry)

# 3. Predict Button (Centered at bottom)
tk.Button(root, text="Predict Monthly Charges", command=predict_price, bg="#0078FF", fg="white", font=("Arial", 11, "bold"), width=25).pack(pady=15)

root.mainloop()

```

Select Model: Decision Tree

gender	male	SeniorCitizen	1
Partner	no	Dependents	no
tenure	32	PhoneService	no
MultipleLines	no phone service	InternetService	no
OnlineSecurity	no	OnlineBackup	no
DeviceProtection	no	TechSupport	yes
StreamingTV	yes	StreamingMovies	yes
Contract	one year	PaperlessBilling	yes
PaymentMethod	credit card	TotalCharges	341
Churn	no		

Predict Monthly Charges

Prediction

 Model: Decision Tree
Monthly Charges: \$19.95

OK

Select Model: KNN

gender	male	SeniorCitizen	1
Partner	no	Dependents	no
tenure	32	PhoneService	no
MultipleLines	no phone service	InternetService	no
OnlineSecurity	no	OnlineBackup	no
DeviceProtection	no	TechSupport	yes
StreamingTV	yes	StreamingMovies	yes
Contract	one year	PaperlessBilling	yes
PaymentMethod	credit card	TotalCharges	341
Churn	no		

Predict Monthly Charges

Prediction ×

Model: KNN
Monthly Charges: \$77.79

Select Model: Linear Regression

gender	male	SeniorCitizen	1
Partner	no	Dependents	no
tenure	32	PhoneService	no
MultipleLines	no phone service	InternetService	no
OnlineSecurity	no	OnlineBackup	no
DeviceProtection	no	TechSupport	yes
StreamingTV	yes	StreamingMovies	yes
Contract	one year	PaperlessBilling	yes
PaymentMethod	credit card	TotalCharges	341
Churn	no		

Predict Monthly Charges

Prediction ×

Model: Linear Regression
Monthly Charges: \$15.28